

75

76

77

78

79

80

81

82

```
resource "aws_s3_bucket" "logs" {
  bucket = "logs"
  acl    = "private"
  policy = jsonencode({
    Version: "2012-10-17",
    Statement: [
      {
        Effect: "Deny",
        Principal: "*",
        Action: "s3:*",
        Resource: "arn:aws:s3:::logs/*",
        Condition: {
          StringEquals: {
            "aws:PrincipalOrgID": "o-1234567890"
          }
        }
      }
    ]
  })
}
```

2) What happens to Terraform if someone changes infra manually from AWS Console?

Answer:

- Terraform will **detect drift during terraform plan** and show the difference between actual and expected state.
- On terraform apply, it may **overwrite** the manual changes to match the code.
- Use terraform refresh or plan often to stay in sync with real infra.

Drift means the actual infrastructure has **changed outside Terraform**, so it no longer matches the state file.

3) What is the use of modules in Terraform?

Answer:

- Modules let you **reuse Terraform code** (like functions in programming) for common infra components (e.g., VPC, EC2, S3).
- Helps in **organizing** code, improving **reusability**, and **reducing duplication**.

76

77

78

79

80

81

82

83

```
hcl
# main.tf
module "ec2_instance" {
  source = "../modules/ec2"
  instance_type = "t2.micro"
  ami_id       = "ami-12345678"
}

hcl
# modules/ec2/main.tf
resource "aws_instance" "this" {
  ami           = var.ami_id
  instance_type = var.instance_type
}
```

4) What is the use of the "terraform taint" command?

Answer:

"terraform taint" marks a resource as tainted, meaning it will be destroyed and recreated during the next apply.

Use this if a resource is faulty or inconsistent, but its configuration hasn't changed.

Example:

```
"terraform taint aws_instance.example"
```

This helps fix broken resources without manually deleting them.

☒ When do you use terraform taint?

1. **A resource is faulty**
Example: An EC2 instance was created, but the app inside it isn't working. Instead of debugging, you decide to recreate it.
2. **You want to refresh the resource**
Example: A manual change was made to the resource (diff) and you want Terraform to recreate it in the next apply.
3. **Testing purposes**
You want to test provisioning from scratch for a particular resource.



Search



ENG
IN



12:16
17-05-2026

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

File View Tools Help

Share

77

78

79

80

81

82

83

84

5) Explain the concept of "null_resource" in Terraform.

• Answer:

```
resource "null_resource" "example" {
  provisioner "local-exec" {
    command = "echo Hello from null_resource"
  }

  triggers = {
    always_run = "${timestamp()}"
  }
}
```

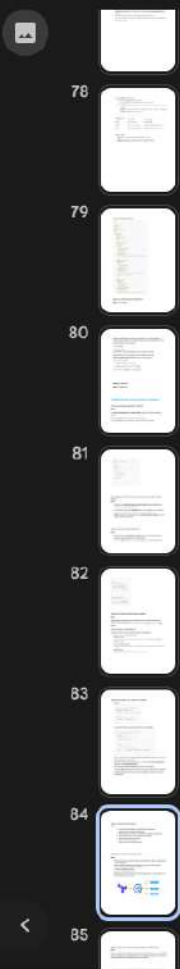
Use Case: SSH into an existing EC2 instance and install Nginx

```
resource "null_resource" "remote_install_nginx" {
  provisioner "remote-exec" {
    inline = [
      "sudo apt update",
      "sudo apt install -y nginx"
    ]
  }

  connection {
    type        = "ssh"
    user        = "ubuntu"
    private_key = file("~/ssh/priv-key.pem")
    host        = "54.12.34.56" # Replace with your EC2 public IP
  }
}
```

Null_resource lets you run **local scripts or commands** using provisioner blocks (like local-exec or remote-exec).

- It doesn't create any real cloud resource — used for **custom logic, triggering actions**, or **provisioning manually**.
- It is commonly used with "triggers", which are key-value pairs. When a trigger changes, the null_resource re-runs the script. Useful for tasks like registering DNS, restarting services, or calling APIs after infra creation. triggers ensure the null_resource runs when inputs change (or every time if you use timestamp()).



6) What are Terraform best practices?

Answer:

- ✓ Use remote backends (S3 + DynamoDB) for state storage.
- ✓ Use modules for reusable infrastructure.
- ✓ Never hardcode secrets, use environment variables or secret stores.
- ✓ Pin provider versions to avoid unexpected changes.
- ✓ Use "terraform plan" before apply.
- ✓ Keep ".tfvars" files out of Git.
- ✓ Use automation via CI/CD pipelines.

7) Explain the concept of Terraform Workspace.

Answer:

- Workspaces help you create multiple environments (dev, staging, prod) using the same configuration.
- Each workspace maintains its own state file, so resources don't conflict.
- You can create and switch workspaces using:
 - "terraform workspace new dev"
 - "terraform workspace select prod"
- Workspaces are best for simple environment separation. For complex infra, use different folders or repositories.



8) What happens first — infrastructure provisioning or state file update?

Answer:

Terraform always provisions the infrastructure first. Once the resources are successfully created or updated, it then updates the state file to reflect the latest status. So, state file entries only appear after successful provisioning, not before.

9) How do you handle secrets in Terraform? Or How will you handle sensitive data in terraform?

Answer:

```
variable "db_password" {  
  type      = string  
  sensitive = true  
}
```

Pass using env variable
export TF_VAR_db_password="my-secret"

❗ **Never hardcode secrets** in .tf files or terraform.tfvars.

❗ Use **environment variables, Terraform Cloud workspaces, or secret managers** (like AWS Secrets Manager or Vault).

❗ You can also use sensitive = true in variables to avoid secrets being shown in output.

Use:

- ✓ AWS SSM Parameter Store or Secrets Manager
- ✓ Vault by HashiCorp
- ✓ Mark variables as "sensitive" in Terraform to mask them in logs. Also, encrypt your remote state file, and ensure proper IAM policies to restrict access.

10) What are different ways to store sensitive information in Terraform execution?

Answer: Same as Q9

...

11) How can you use details of manually created resources in Terraform?

Answer:

- Use data blocks (data sources) in Terraform to read existing resources created outside Terraform.
- Do not use resource block — it will try to create the resource again.

```
data "aws_vpc" "existing" {
  filter {
    name   = "tag:Name"
    values = ["my-existing-vpc"]
  }
}

output "vpc_id" {
  value = data.aws_vpc.existing.id
}
```

...

12) You've provisioned infra using CloudFormation. Now you want to move to Terraform. What do you do about state file?

Answer:

- Use terraform import to bring existing resources under Terraform control without recreating them.
- After import, write matching code (.tf files) for each imported resource manually.

81

82

83

84

85

86

87

88

```
terraform import aws_instance.my_ec2 i-bab1234b756789

Then in your .tf file:

resource "aws_instance" "my_ec2" {
  # must match exactly with imported resource
}
```

✓ This avoids breaking the existing infra and builds the correct state file.

13) Advantages and Disadvantages of Terraform vs CloudFormation?

Answer:

Terraform Multi-cloud, readable syntax (HCL), reusable modules Not AWS-native, state management is manual (unless using Terraform Cloud)

CloudFormation Deep AWS integration, drift detection, native support Only works with AWS, JSON/YAML can be complex and verbose

Use Terraform for multi-cloud, use CloudFormation for tight AWS-only environments.

✓ **Terraform Advantages**

- Multi-cloud support (AWS, GCP, Azure etc.)
- Modular and reusable code (easy use of modules)
- Better CLI and community support

✗ **Terraform Disadvantages**

- No official rollback if apply fails
- State file management is your responsibility
- Not AWS-native (you may need custom modules for some AWS features)

✓ **CloudFormation Advantages**

- AWS native — tightly integrated with AWS services
- Drift detection, rollback, stack policies built-in
- No need to manage state file manually

✗ **CloudFormation Disadvantages**

- AWS only (no multi-cloud support)
- YAML/JSON templates are harder to reuse
- Slower evolution of features than Terraform

Let me know if you want a quick comparison table too.

14) I have VPC and EC2 modules. Both called at once. How does EC2 get VPC ID at runtime?

Answer:

- Use **output** from the VPC module and **pass it as input** to the EC2 module.
- Terraform builds a **dependency graph** and ensures VPC is created before EC2.

```
# VPC module
output "vpc_id" {
  value = aws_vpc.vpc.id
}

# Main file
module "vpc" {
  source = "../modules/vpc"
}

module "ec2" {
  source = "../modules/ec2"
  vpc_id = module.vpc.vpc_id # dependency created here
}
```

Terraform handles the **order of execution** automatically.

15) What do you understand by provisioner in Terraform?

Answer:

```
resource "aws_instance" "web" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"

  provisioner "local-exec" {
    command = "echo Instance created!"
  }
}
```

Provisioners let you execute scripts or commands on the target machines after resource creation. Types include:

- * **"local-exec"**: Runs a script locally on your machine.
- * **"remote-exec"**: Runs a script on the remote server (like EC2). They are useful for bootstrapping, like installing agents or setting up config. However, provisioners are considered last resort, as they break the declarative model and may lead to unpredictable behavior.

83

84

85

86

87

88

89

90

16) How can you destroy a specific resource in Terraform?

Answer:

Use the "-target" flag:

```
"terraform destroy -target=aws_instance.myec2"
```

This only deletes the specified resource, not the whole infrastructure. Useful for debugging or replacing specific resources safely.

17) How to see complete details of a resource?

Answer:

Use:

```
"terraform state show <resource>"  
terraform state show aws_instance.my_ec2
```

It displays all current values in the state file like IDs, tags, IPs, AMIs, etc. Great for debugging and understanding what Terraform is managing.

18) Use case of dynamic block in Terraform?

84

85

86

87

88

89

90

91

Answer:

```
variable "ports" {
  default = [80, 443]
}

resource "aws_security_group" "web_sg" {
  name = "web-sg"

  dynamic "ingress" {
    for_each = var.ports
    content {
      from_port = ingress.value
      to_port   = ingress.value
      protocol = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}
```

Use "dynamic" blocks when you need to create repeating nested blocks based on a list or variable.
Example: Security group rules, IAM policies, tags. This reduces repetition and makes configurable.

19) What are some common Terraform functions?

- String: "join()", "split()", "format()", "replace()"
- Collection: "length()", "contains()", "lookup()"
- Numeric: "min()", "max()", "ceil()"
- File/Template: "file()", "templatefile()"
- Encoding: "base64encode()" These help in customizing resource creation and logic flow.

20) How do you prevent a resource from being deleted in Terraform?

85

86

87

88

89

90

91

92

Answer:

```
resource "aws_s3_bucket" "critical" {
  bucket = "my-important-bucket"

  lifecycle {
    prevent_destroy = true
  }
}
```

Useful for critical resources like S3 buckets, DBs, or IAM roles.

Use the "lifecycle" block:

- Use lifecycle block with `prevent_destroy = true` inside the resource block.
- Terraform will **throw an error** if someone tries to delete that resource.

21) What is the difference between count and for_each in Terraform?

```
resource "aws_instance" "ec2" {
  count = 2
  ami = "ami-123"
  instance_type = "t1.micro"
}
```

```
resource "aws_s3_bucket" "buckets" {
  for_each = toset(["a", "b"])
  bucket = "bucket-${each.key}"
}
```

- count is used for repeating identical resources (by index).
- for_each is used for resources with unique keys (map or set).
- count gives numeric index (count.index), for_each gives item (each.key / each.value).

22) What is a data block in Terraform?

- It lets you fetch existing resources (read-only).
- You don't create resources with it.
- Example: Get current AMI ID or VPC ID to use in a new resource.

86

87

88

89

90

91

92

93

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners     = ["099720199477"]
  filter {
    name   = "name"
    values = ["ubuntu/images/*"]
  }
}
```

23) How do you use terraform apart from initial infra setup for a client ?

Answer:

We generally use Terraform to create infrastructure for new customers or clients. But we also keep improving our existing Terraform modules or add new modules for services like RabbitMQ, Redis, ElastiCache, Lambda, or any other services that were not there earlier in our code.

We also use TF to add new resources in an existing customer's infrastructure, we simply update the tfvars file with the required values, like adding a new S3 bucket or a new subnet.

Also, If a customer project ends, we delete their infrastructure using Terraform destroy, so that unused resources are properly cleaned up and billing is reduced.

DOCKER Interview Questions & Answers

1) You are writing a Dockerfile where you want to pass a build-time variable (like version of Java) and also set an environment variable inside the container (like JAVA_HOME). How would you decide when to use ARG vs ENV? OR What is the difference between "ARG" and "ENV"?

87

88

89

90

91

92

93

94

Answer:

```
# Use official base image
FROM node:18

# Set build-time variable
ARG APP_VERSION=1.0
ENV VERSION=$APP_VERSION

# Set working directory
WORKDIR /app

# Copy files
COPY package*.json ./
RUN npm install
COPY . .

# Expose port
EXPOSE 3000

# Default command
CMD ["node", "app.js"]
```

Feature	ARG	CMD
Scope	Build-time	Run-time
Accessibility	Not available in container	Available during container start
Default Value	Can set default via <code>ARG name=val</code>	Can set default via <code>CMD ["..."]</code>
Override	Override with <code>--build-arg</code>	Override with <code>docker run</code>
Use Case	Set env at build (e.g., version)	Default command to run

* "ARG" is used to define variables that are available only during image build time. It cannot be accessed after the container is running.

* "ENV" defines environment variables that persist in the final image and can be accessed both during build and runtime. It is stored as metadata.

* Example:

1. "dockerfile ARG VERSION=1.0 ENV APP_ENV=prod "

Notes:
Metadata is **part of the image**, but not inside filesystem layers.

2. The container reads metadata at **startup** to know:
 - Which command to run (CMD/ENTRYPOINT)
 - Which environment variables to set (ENV)
 - Other runtime settings
3. You **don't need the Dockerfile** inside the image —the metadata is enough for the container to start correctly.
4. • **Layers** = filesystem snapshots (tar files / diff directories)
5. • **Metadata/config** = JSON files describing the image, layers, and runtime info.
We can see metadata of image with command – `sudo docker inspect image-name`

2) What is the difference between "RUN" and "CMD"? OR You are building a Docker Image. During build, you need to install dependencies (e.g., `apt-get install`) and after container starts, you want it to run `python app.py` automatically. Which instructions (RUN or CMD) should you use in each step and why?

Answer:

1. Dockerfile for a Python Flask App

```
Dockerfile

FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .

# RUN installs dependencies (useful here!)
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "app.py"]
```

2 Example requirements.txt

```
txt

flask==2.3.3
requests==2.31.0
gunicorn
```

Note: requirements.txt is a **dependency list file** for Python applications. It tells pip which python libs are needed which versions. Without docker, we can run the app

Page 95 / 158

89

90

91

92

93

94

95

96

simply by command "python app.py" but it should have all dependencies installed on the server.

- "RUN": Executes a shell command during the build of the Docker image and creates a new layer.
RUN apt-get update && apt-get install -y nginx
- "CMD": Specifies the default command to run when a container starts from the image. This command will run **every time** the container is started, restarted, or recreated — because it's the container's **main process**.
CMD ["python", "app.py"]
- "RUN" is for build-time setup; "CMD" is for runtime behavior.

3) You have a Docker image for a database backup script. Sometimes you want to run it with default parameters, and other times you want to pass custom arguments at runtime. Would you choose CMD or ENTRYPOINT in the Dockerfile, and why? OR What is the difference between "CMD" and "ENTRYPOINT"?

Answer:

Case 1: Only CMD

Dockerfile

```
FROM ubuntu
CMD ["echo", "hello from CMD"]
```

Command	What Actually Runs	Result / Output
docker run --image=web	echo Hello From CMD	Hello From CMD
docker run --image=web "HE"	HE (replaces CMD)	Error: exec: "HE": executable file not found
docker run --image=web echo "HE"	echo HE	HE

Key Rule:

- CMD is fully replaced by any args passed in docker run.

90

91

92

93

94

95

96

97

Case 2: Only ENTRYPOINT

Dockerfile:

```
FROM ubuntu
ENTRYPOINT ["echo", "hello from ENTRYPOINT"]
```

Command	What Actually Runs	Result / Output
docker run image-name	echo hello from ENTRYPOINT	hello from ENTRYPOINT
docker run image-name "HEL"	echo HEL from ENTRYPOINT HEL	HEL from ENTRYPOINT HEL
docker run image-name echo "HI"	echo HEL from ENTRYPOINT echo HI	HEL from ENTRYPOINT echo HI

Key Rule:

- ENTRYPOINT always runs
- Arguments passed at docker run are appended to ENTRYPOINT

Case 3: ENTRYPOINT + CMD

Dockerfile:

```
FROM ubuntu
ENTRYPOINT ["echo", "hello"]
CMD ["from CMD"]
```

Command	What Actually Runs	Output
docker run image-name	echo hello from CMD	hello from CMD
docker run image-name "HI"	echo hello HI	hello HI
docker run image-name echo "HEL"	echo hello echo HEL	hello echo HEL

ENTRYPOINT is fixed, CMD is default arg and is replaced if you pass anything via docker run.

```
cat > /dev/null 172.17.0.130 docker-cmd-entrypoint-demo:1 cat Dockerfile
FROM ubuntu
ENTRYPOINT ["echo", "hi from entrypoint "]
CMD ["hi from cmd"]
cat > /dev/null 172.17.0.130 docker-cmd-entrypoint-demo:1
cat > /dev/null 172.17.0.130 docker-cmd-entrypoint-demo:1
cat > /dev/null 172.17.0.130 docker-cmd-entrypoint-demo:1 vs Dockerfile
cat > /dev/null 172.17.0.130 docker-cmd-entrypoint-demo:1 docker run ubuntu:latest echo hi from xiaohu
hi from xiaohu
cat > /dev/null 172.17.0.130 docker-cmd-entrypoint-demo:1
```

How to override entrypoint syntax-
docker run --entrypoint="main-command" image-name "arguments"

```
cat > /dev/null 172.17.0.130 docker-cmd-entrypoint-demo:0 docker run --entrypoint=echo ubuntu:latest hello
hello
cat > /dev/null 172.17.0.130 docker-cmd-entrypoint-demo:1
```

- If used alone, both of them are used to run commands when container starts.
- "CMD": Can be overridden when running a container. **CMD is fully replaced** by any args passed in docker run.

97 / 158

75%

91

92

93

94

95

96

97

98

- "ENTRYPOINT": Treats its values as the executable and cannot be overridden easily. ENTRYPOINT always runs, it is fixed. Arguments passed at docker run are treated as additional arguments to ENTRYPOINT.
- Together: Entrypoint is the main command while CMD has the arguments and args passed to docker run will override CMD, entrypoint will stay fixed.
- "dockerfile ENTRYPOINT ["python3"] CMD ["app.py"] "

This always runs: "python3 app.py"

4) What do you understand by tag immutability in Docker? OR Your team pushes Docker images with the tag latest. One day, you notice that yesterday's latest is different from today's latest. What does this mean and why can it be a risk in production deployments?

Answer:

- ✓ In Docker, tags are changeable[mutable/repeatable]. Like in question, yesterday code pushed and tag was latest, today also code pushed, tag is again latest. So same tag can be used to refer to different code.
- ✓ Tag immutability means once an image is pushed with a specific tag like "v1.0", that tag should never be overwritten. This we should follow, it is best practise. So that if yesterday code pushed with v1 tag, today code pushed with v2 then v3 and so on. So in case of some issue, we can revert back to some old tag [old code].
- ✓ This ensures reliable, reproducible deployments in CI/CD, avoiding image conflicts and ambiguity.

In prod env, you should make tags as immutable. **Immutable tag** = once an image is pushed with a tag (e.g., v1.0.0), **it cannot be changed or overwritten**. Helps prevent "it worked yesterday, now it's broken" due to a tag pointing to a different image today.

- ✓ Docker tags are changeable[mutable/repeatable].

5) Can you make latest tag immutable? OR in your company, someone asks: "Can we make the latest Docker tag immutable so nobody can overwrite it accidentally?" How would you respond?

Answer:

Page 98 / 158

72

93

94

95

96

97

98

99

Read above answer first. Yes — but only at the registry level, not in Docker itself.

Docker itself treats all tags mutable (can be overwritten)— including latest — as just labels pointing to an image digest. It doesn't care if it's latest, v1.0, or stable.

- If you want to make latest immutable in ECR, Go to your ECR repository settings.
- Set "Image Tag Mutability" to IMMUTABLE.

This applies to all tags, including latest. Now, If you try to push a new image to the same tag (like latest), it will fail with error : denied: The image tag 'latest' is immutable

Summary

Registry	Can make (latest) immutable?	How to do it
AWS ECR	Yes	Set repo to IMMUTABLE
Docker Hub	Not directly	Use digests / customizing
GHCR	Not directly	Control via RULE rules
Helfer	Yes	Tag immutability policies

...

6) Why is a container called lightweight? OR Why containers are lightweight
Why you prefer Docker containers over full VMs when deploying microservices.
How do you explain why containers are called lightweight?

Answer:

- No need for a full OS per container.
- Shares the host's kernel.
- Starts in seconds and uses minimal memory, unlike VMs.

...

7) What is the use of multi-stage Docker builds? OR You are building a Java application where you need Maven to build the JAR, but Maven itself should not be inside the final image. How can you write the Dockerfile to achieve this?

93

94

95

96

97

98

99

100

Answer:

Minimal Multi-Stage Dockerfile (Maven build → JAR run)

```
Dockerfile

# ---- Stage 1: Build with Maven ----
FROM maven:3.9.6-eclipse-temurin-17 AS builder

WORKDIR /app
COPY pom.xml .
COPY src ./src

# Build the JAR (skip tests to speed up)
RUN mvn clean package -o skip

# ---- Stage 2: Runtime ----
FROM eclipse-temurin:17-jdk-slim

WORKDIR /app

# Copy only the final JAR file from target/
COPY --from=builder /app/target/*.jar app.jar

# Run the JAR
CMD ["java", "-jar", "app.jar"]
```

- Helps reduce final image size by separating build and runtime environments.
- First stage builds the app, second stage copies only necessary files.
- Example:
- "dockerfile FROM golang as builder WORKDIR /app RUN go build -o main .
- FROM alpine COPY --from=builder /app/main/main CMD ["/main"] "

8) What is the default networking in Docker? OR You run two containers without specifying any network in Docker. By default, how do they communicate with each other and what kind of network are they placed in?

Answer:

* The "bridge" network is default. All containers get an internal IP and can communicate via this bridge.

9) "Is a container a mini VM? How does it work internally?" How would you explain what a container really is?

94

95

96

97

98

99

100

101

Answer:

- A container is an isolated environment that includes the app and its dependencies, running on the host OS. Running instance of a docker image is called container.
- It shares the host kernel and isolates processes, filesystems, and networking.

10) In your Dockerfile, you want to copy a local app.jar file into the image, but in another case you want to download a tar file from a URL and extract it automatically. Would you use COPY or ADD, and why? OR What is the difference between "ADD" and "COPY"?

Answer:

- Both copy files, but "ADD" has extra features:
- Can download from URL
- Can extract ".tar" files
- Use "COPY" for basic file transfer for clarity and performance.

11) Can you delete the image from which a container is running? OR One of your teammates tries to delete an image, but Docker shows that it's still being used by a running container. Why does Docker prevent deletion, and what options do you have if you still want to remove it?

Answer:

No — even with force, Docker won't allow deleting image for running container.

Docker **prevents deletion** of any image that is currently being **used by a container** — when that container is **running**. **With force you can untag the repository name with image id.**

But if you stop the container, then you can forcefully delete the image with image id.

`docker rmi -f my-image-id`

DEMO SCREENSHOT FROM LIVE SESSION:

95

96

97

98

99

100

101

102

A container[6027] is running from the image 9402.

```
root@root-p:~# docker ps --filter "name=flask-app" --format "{{.ID}}\t{{.ImageID}}\t{{.Status}}\t{{.Ports}}\n"
6027\t9402\tUp 30s\t0.0.0.0:5000->5000/tcp
```

Two pointers[repo name] pointing to same image[same image id 9402]. This is actually one image only. We cant delete the image [by image id]. But we can untag the image[remove the repository names pointers].

```
root@root-p:~# docker rmi $(docker ps --filter "name=flask-app" --format "{{.ImageID}}")
Error response from daemon: conflict: unable to remove repository reference "flask-app" (not found) - container 6027 still has it
```

Second repository pointer also deleted. But image is still there. With image id. This is dangling image.

```
root@root-p:~# docker images --filter "dangling=true" --format "{{.ID}}\t{{.ImageID}}\t{{.Repository}}\n"
9402\t9402\t<none>
```

After this forcefully also, if you try delete image by image id , it will not delete because a container is running from it.

```
root@root-p:~# docker rmi 9402
Error response from daemon: conflict: unable to remove repository reference "9402" (not found) - container 6027 still has it
```

We stop the container but do not delete the container.

12) What will happen to your container if you delete an image from which container was created?

Answer:

- If container is still running — it will continue. (If you have just untagged your image from repo) and it can be restart also.
- If stopped — the container remains but we can restart that container. We can't create new containers from that image unless you re-pull it.

13) What happens if you restart a container after deleting its image?

Answer:

If we have deleted the image by image-id, we won't be able to create container again. And we cannot delete image without stopping container.

If we have untagged the image with repo, then image is still there on our server. We can restart the container and if it was running, it will continue to run. We can untag image while container was in running state.

But if you delete the container and try to recreate it, the image must exist — otherwise it will fail.

14) Write a Dockerfile to start nginx server

Answer:

Dockerfile

Using base image nginx. Copies from working directory "/index.html" and paste it inside image at path "/usr/share/nginx/html/". Container will run using container port 80 with command "nginx -g daemon off"

```
FROM nginx:latest
COPY ./index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

15) Docker vs VM virtualization.

Answer:

- Docker shares the host OS kernel, while VM has its own OS for each instance.
- Docker is lightweight and faster, while VMs are heavier and slower.
- Docker starts in seconds, while VMs take minutes to boot up.
- Docker provides process-level isolation, while VMs provide full system isolation.
- Docker consumes fewer system resources, while VMs consume more due to full OS.
- Docker containers are highly portable, while VMs are less portable.
- Docker is ideal for microservices and CI/CD, while VMs are better for full OS environments or legacy apps.

16) How to run Docker containers as non-root user? OR Your security team asks you not to run containers as root. How can you modify your Dockerfile or container startup, so the app runs as a non-root user?

Answer:

- Add this line in the dockerfile and then use user appuser in dockerfile.
RUN groupadd -r appgroup && useradd -r -g appgroup appuser
USER appuser
- Explanation:
groupadd -r appgroup: Creates a system group appgroup.
useradd -r -g appgroup appuser: Creates a system user appuser belonging to appgroup.
USER appuser: Ensures all further instructions (RUN, CMD, etc.) run as appuser.

17) How do you mount a volume in Docker? OR You have a database container and want to persist data even if the container is deleted. How would you run the container with a mounted volume?

Answer:

* Using "-v" or "--mount":

105 / 158

75%

99

100

101

102

103

104

105

106

" docker run -v /host/data:/container/data nginx"

docker run -v "path-on-host:path-inside-container"

Running a docker container for Jenkins with volume:

```
docker run -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home jenkins/jenkins:its-jdk21
```

Here "jenkins_home" is the volume name on the host machine and it is mapped to path "/var/jenkins_home" and "jenkins/jenkins:its-jdk21" is the image with tag.

...

18) Types of Docker networking. OR Your colleague is confused about networking in Docker. They ask: "When should we use bridge, host, or overlay network modes?" How would you explain each with use-cases?

Answer:

- ✓ Bridge: Default network
- ✓ Host: Uses host network directly
- ✓ Overlay: Cross-host networking for Swarm. Overlay used in cases where containers are on different hosts. Like in docker swarm or k8
- ✓ None: No network assigned to the container
- ✓ Macvlan: Assigns MAC address and appears on physical network

...

19) Command to run a container. OR You want to run an Nginx container, map port 8080 on your host to 80 inside the container, and give it a custom name mynginx. What Docker command would you use?

Answer:

Syntax - docker run -d -p host-port:container-port container-name image-name

Example - "docker run -d -p 80:80 --name my-app nginx"

-d used to run process in background

-p used for port where first port is port of the host machine on which container is running (first port is our port) and second port is the port of the container.

--name my-app this is used to give name to our container otherwise some random name will get assign by docker.

...

20) Dockerfile best practices to reduce size:

Answer:

- Use Alpine or slim base images
- Combine multiple "RUN" statements with &&. It will reduce the layers in image. Instead of this one,

example (bad):

```
dockerfile
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*
```

4. Why && in a single RUN is better?

Optimized version:

```
dockerfile
RUN apt-get update && \
  apt-get install -y curl && \
  rm -rf /var/lib/apt/lists/*
```

Multiple RUN instructions create multiple layers; temporary files remain in earlier layers even if deleted later, so combining commands in a single RUN keeps the final image small.

- Clean up cache and temp files
- Use multi-stage builds
- Avoid unnecessary layers and debug tools

21) What is virtualization?

Answer:

Virtualization means running multiple virtual computers on one physical machine. Imagine you have one big computer, but you divide it into smaller pretend computers — each one thinks it's its own machine, with its own CPU, memory, disk, etc.

These "pretend" computers are called Virtual Machines (VMs).

Example, You have one laptop, but you install software (like VirtualBox or VMware) and run:

Windows in one virtual machine

Ubuntu in another

Kali Linux in a third



Search



ENG
IN

12:17
17-05-2026



101



102



103



104



105



106



107



108



All at the same time, on the same physical laptop.
Need of virtualisation:
Run **multiple OSES** on one machine
Save hardware cost
Easily **test and develop** different environments
Improve **server utilization** in data centers

22) What's difference between k8 and Docker in terms of their usage and if any? We already use Docker, why do we need Kubernetes?

Answer:

Feature	Docker (Standalone)	Kubernetes
Purpose	Containerization (build, ship, run containers)	Container orchestration (manage multi-container apps)
Scope	Runs single or few containers	Manages clusters of containers
Scaling	Manual (docker run multiple times)	Auto-scaling, rolling updates, self-healing
Networking	Basic bridge/host	Advanced service discovery & DNS
Deployment	Manual via CLI or Compose	Declarative via YAML (Deployments, Services, etc)
Storage	Limited options (volumes, bind mounts)	Supports persistent volumes, storage classes
Ideal for	Small apps or dev/test environments	Production-grade, distributed apps

23. Suppose I am a running monolithic application on an AWS EC2 machine. I want to containerize it. What will be all the steps to do achieve the same?

Answer:

1. ssh to ec2, install docker.
2. Write dockerfile
3. Build docker image and run container from it.
docker build -t my-monolith-app .
docker run -d -p 80:8080 my-monolith-app

24). Write a Docker file for java application deployment for which source repo is on GitHub. OR You are tasked with writing a Dockerfile to deploy a Java application.



Search



ENG

IN



12:17

17-05-2026

The source code is in GitHub, and you need to build it and run it. How would your Dockerfile look?

```
# ----- STAGE 1: Build the application -----
FROM maven:3.9.6-eclipse-temurin-17 AS builder

# Set working directory
WORKDIR /app

# Copy source code
COPY . .

# Build the application (JAR)
RUN mvn clean package -DskipTests

# ----- STAGE 2: Run the application -----
FROM openjdk:17-jdk-slim

# Set working directory
WORKDIR /app

# Copy only the built JAR from previous stage
COPY --from=builder /app/target/*.jar app.jar

# Run the JAR
ENTRYPOINT ["java", "-jar", "app.jar"]
```

25) What is Docker Compose and where do we use it? OR You have an application with three containers: an API, a database, and a cache. Instead of running each with separate docker run commands, how can you manage and run them together



103

104

105

106

107

108

109

110

easily?

```
version: '3'
services:
  web:
    image: nginx
    ports:
      - "80:80"
  redis:
    image: redis
```

Run using:

```
docker-compose up -d
```

1. Docker Compose helps run multi-container applications.
2. You define services, volumes, and networks in a single docker-compose.yml.
3. Useful for local development, testing, and isolated environments.
4. Common for web apps with DB, Redis, etc., running together.

26) What happens when you push the same Docker tag again (e.g., latest)?

1. By default, it overwrites the old image.
2. This causes confusion in CI/CD — deployments might use an unintended image.
3. That's why we avoid reusing tags — this is called **tag immutability**.

Best practice: Use versioned tags (v1.0.1) and avoid re-pushing the same tag.

27) What is Docker Swarm and how is it different from Docker standalone?

1. **Docker Swarm** is Docker's native tool for **container orchestration** (like Kubernetes but simpler).
2. It lets you manage a **cluster of Docker nodes** (Swarm).
3. You can **deploy services**, scale containers, and ensure high availability.
4. Uses docker service and docker stack commands instead of docker run.
5. Swarm handles **load balancing**, **health checking**, and **auto rescheduling** of failed containers.

Commands:

```
docker swarm init
docker service create --name web --replicas 3 -p 80:80 nginx
```



Search



ENG
IN

12:17
17-05-2026

104

105

106

107

108

109

110

111

This runs 3 nginx containers across Swarm nodes.

28) Why does RUN create a new layer?

When Docker executes:

```
docker run nginx
```

Docker:

1. Starts a temporary container from the previous layer
2. Executes the command
3. Takes a **filesystem snapshot**
4. Saves it as a **new immutable layer**

That snapshot includes:

- Files created
- Files modified
- Files deleted (as metadata)

This is why every **RUN** = new layer

29) Why multiple RUN instructions increase image size? How to reduce image size?

Each RUN creates a new immutable image layer. Files added in earlier layers remain even if deleted later, so combining commands in a single RUN avoids persisting temporary files and reduces image size.

Because every RUN instruction creates a new filesystem layer, and deleting files in later layers doesn't remove them from earlier ones.

Multiple RUN instructions create multiple layers; temporary files remain in earlier layers even if deleted later, so combining commands in a single RUN keeps the final image small

✗ BAD Dockerfile (multiple RUNs)

```
Dockerfile
FROM python:3.11-slim

RUN apt-get update
RUN apt-get install -y build-essential gcc
RUN pip install numpy pandas
RUN apt-get remove -y gcc
RUN rm -rf /var/lib/apt/lists/*
```

Gcc is needed in compiling numpy, then it is not needed further at runtime. So we install it use it and then delete it. [numpy/pandas are example libraries. Large parts of numpy are written in C, so gcc is used to compile that C code into binary files

that Python can execute.]
So here Even though gcc was "removed", it still exists in earlier layers + image is heavy.

GOOD Dockerfile (single RUN with &&)

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Here, ✓ gcc was needed during build
✓ gcc is NOT present in final image
✓ apt cache never persisted

30) Why in python Dockerfile, we do only COPY requirements.txt and then later COPY . . and not do copy whole code once like COPY . . ?

Answer:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

In First build:

- requirements.txt copied
- dependencies installed
- app code copied

In Second build (only code change):

- requirements.txt unchanged → cached layer reused ✓

- pip install is skipped
- only app code is copied

SO it is faster. It will make use of cache layers. 2 to 3 second.

✗ Bad Dockerfile (no caching benefit)

```
FROM python:3.11-slim
WORKDIR /app

COPY . .
RUN pip install -r requirements.txt

CMD ["python", "app.py"]
```

What happens on rebuild

- You change one line in app.py
- COPY . . changes
- Docker cache invalidated
- pip install runs again ✗ slow

- If we use COPY . . in the starting, then any change in code will force to rebuild every layer. Take higher time like 2 to 5 minutes. No use of cache and slower builds.

31) To force Docker to rebuild every layer from scratch and not use cache, use this command:

`docker build --no-cache -t my-image .`

32) Example of entrypoint and cmd use case in production where entrypoint remains fixed and cmd can be overridden by developers

```
FROM eclipse-temurin:17-jre

WORKDIR /app

COPY app.jar /app/app.jar

ENTRYPOINT ["java"]
CMD ["-Xms512m", "-Xmx1024m", "-jar", "app.jar"]
```



Search



ENG

IN

12:17

17-05-2026

CICD / JENKINS Interview Questions & Answers

1) What do you understand by CI/CD? OR Your project manager asks you to explain why the team is setting up a CI/CD pipeline and what value it brings to developers and operations. How would you explain what CI/CD means in simple terms?

Answer:

CI = Continuous Integration

CD = Continuous Delivery/Deployment.

In CI, developers frequently merge code into a shared repository, triggering automated builds and tests.

CD ensures that once code is tested, it's automatically delivered to production or staging with minimal manual intervention. It helps release software faster, more reliably, and reduces integration problems.

2) Explain continuous delivery vs continuous deployment. OR Your team currently has automated tests and builds running, but before production deployment, someone manually approves the release. A colleague asks: "Is this continuous delivery or continuous deployment? What's the difference between the two?" How would you answer?

Answer:

- Continuous Delivery: Code changes are automatically built, tested, and pushed to a staging environment, but need **manual approval to deploy to production**.
- Continuous Deployment: Code changes go all the way to production automatically after passing tests. **No manual intervention. Directly deployed to production.**
- The difference lies in the final deployment step — manual vs automatic.

3) What do you understand by Jenkins shared library? OR Your company has 20 Jenkins pipelines, all of which repeat the same steps for code checkout, build, and notifications. How can you avoid repeating code in every pipeline, and what Jenkins feature would you use?

Answer:

- Reusable pipeline code.
- Stored in separate Git repo.
- Use `@Library('my-lib')`.

```
@Library('my-shared-lib') _
helloworld()
```

Shared libraries in Jenkins allow you to reuse common pipeline code across multiple Jenkinsfiles. You define reusable functions or stages in a centralized Git repository and call them in different pipelines using `@Library`. It promotes DRY (Don't Repeat Yourself) and modular code for pipeline automation.

4) What is multi-branch pipeline? OR Your team works with GitHub where developers push changes to multiple branches like dev, qa, and main. You want Jenkins to automatically detect these branches and create pipelines for each. Which Jenkins feature allows this?

Answer:

- Auto detects Git branches.
- Creates job per branch.
- Uses branch-specific Jenkinsfile.

A multibranch pipeline automatically detects branches in a repository and creates pipelines for each one. Jenkins scans the repo and creates jobs for `dev`, `feature/`, `main`, etc. It's ideal for GitHub workflows and helps test each branch independently using its own Jenkinsfile.

Benefits:

- Automatic discovery of branches and PRs.

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

File View Tools Help

Share

109

110

111

112

113

114

115

116

No manual job creation per branch.

Sample Jenkinsfile (same for each branch):

```
groovy
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        echo "Building branch: ${env.BRANCH_NAME}"
      }
    }
    stage('Test') {
      steps {
        echo "Testing on branch: ${env.BRANCH_NAME}"
      }
    }
  }
}
```

5) How will you setup notification system for your pipeline? OR Your manager wants to be notified whenever a Jenkins pipeline fails. How would you configure Jenkins to send notifications (e.g., via email or Slack) when a build breaks?

Answer:

A job added in github actions to trigger notification to google chat group.

```
# workflow complete notification
name: Notification Github Actions to Google Chat
uses: thailana-qe/githubactions-gogicChat@
with:
  name: Tests E2E
  url: "https://chat.googleapis.com/v1/spaces/AAAAH2UPto/messages?key=Akz
  status: ${ job.status }
  if: always()
  # if: failure() # Only triggers if the job fails
  continue-on-error: true
```

You can integrate Slack, Google Chat, Email, or any other tool using plugins. For Slack, install the Slack plugin, configure a webhook, and use "slackSend()" in your pipeline to send build status notifications. Notifications can be customized in the "post" section of

the Jenkinsfile.

- For static:
 - Install plugin.
 - Add webhook URL.
 - Use `slackSend()`.

Snippet:

```
groovy
def s {
    slackSend(channel: 'general', message: "Build done: ${env.TASK_NAME}")
}
```

6) How do you integrate Jenkins with GitHub to trigger builds on push? OR Your team wants Jenkins builds to trigger automatically whenever someone pushes code to GitHub. How would you integrate Jenkins with GitHub to achieve this?

Answer:

- Install the GitHub Integration plugin in Jenkins
- Generate a GitHub webhook pointing to Jenkins URL: `"http://<jenkins-url>/github-webhook/"`
- Enable GitHub hook trigger in job settings. Jenkins will automatically trigger a job when a push or PR is made to GitHub.

7) What do you understand by declarative and scripted pipeline? OR You inherit two Jenkinsfiles: one is written in declarative syntax, the other in scripted syntax. A teammate asks: "What's the difference between these two styles, and when should we use one over the other?" How would you explain it?

Answer:

Feature	Declarative	Scripted
Syntax	<code>pipeline {}</code> block	Groovy code
Simplicity	easy, opinionated	flexible, complex
Use case	Most pipelines	Custom logic needed

111

112

113

114

115

116

117

118

- Declarative pipeline uses a structured syntax ("pipeline {}") and is simpler, suitable for most use cases.
- Scripted pipeline is based on Groovy and offers more flexibility, useful for complex workflows. Declarative is recommended for maintainability; scripted is powerful but complex.

...

8) Can we add an external EC2 machine to a running Jenkins master? OR Your Jenkins master is already running on one EC2 instance, but you need extra build capacity. A teammate suggests adding another EC2 machine as a Jenkins node. How would you connect that external EC2 machine to the existing Jenkins master?

Answer:

Yes, you can configure an EC2 as a Jenkins agent/slave. Install Java and Jenkins agent on the EC2 machine and connect it to Jenkins master using SSH or JNLP. Configure in Jenkins → Nodes.

Enables parallel builds.

This helps in scaling builds across different environments.

...

9) How do you manage sensitive information in Jenkins? OR In your pipeline, you need to use API keys and database passwords. A developer suggests hardcoding them in the Jenkinsfile. How would you handle this situation and manage sensitive information securely in Jenkins?

Answer:

```
withCredentials([string(credentialsId: 'my-token', variable: 'TOKEN')]) {  
    sh 'echo $TOKEN'  
}
```

Use Jenkins credentials plugin to store secrets like passwords, tokens, and SSH keys. You can access them securely in pipelines using "credentialsId". Avoid hardcoding secrets directly in the Jenkinsfile.

- ☑ Use **Credentials Plugin**.
- ☑ Store secrets securely.
- ☑ Types: secret text, SSH key, username/password.

10) How do you manage different permissions for different users in Jenkins? OR
Your company has multiple teams using Jenkins. Developers should only run and view their jobs, while admins should manage configurations and plugins. How do you manage these different permissions in Jenkins?

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

File View Tools Help

Share

113

114

115

116

117

118

119

120

Jenkins - Manage and Assign Roles

Global roles

User/group admin, developer

User/group

Anonymous

User/group to add

Item roles

User/group

Anonymous

User/group to add

Node roles

User/group

Anonymous

User/group to add

Save Apply

Use Role-Based Access Control (RBAC) plugin. It allows you to create roles (like admin, dev, QA) and assign permissions like job create/view/build accordingly. Users/groups can then be mapped to roles, offering fine-grained access control.

---4

11) I want my pipeline to continue even if one stage fails — how? Or . If any stage in the pipeline fails, how to continue remaining stages?

Answer:

- Use "catchError" or "script" block with "try-catch".

In Jenkins Declarative Pipeline, use post, always, or catchError.

114

115

116

117

118

119

120

121

For example:

```
stage('Test') {
  steps {
    catchError(buildResult: 'SUCCESS', stageResult: 'FAILURE') {
      sh 'exit 1'
    }
  }
}

stage('Build') {
  steps {
    script {
      try {
        sh 'exit 1'
      } catch (err) {
        echo "Ignoring failure"
      }
    }
  }
}
```

This way, the pipeline won't stop and will move to the next stage.

12) How do you cleanup temporary files in Jenkins during runtime?

Answer:

Snippet 1: Declarative

```
groovy

post {
  always {
    cleanWs()
  }
}
```

Snippet 2: Scripted

```
groovy

stage('Cleanup') {
  steps {
    sh 'rm -rf /tmp/my-temp-files'
  }
}
```

Use cleanWs() step.

Use sh 'rm -rf <path>' in script block.

Run in post block or cleanup stage.

Clean workspace before/after build.

115

116

117

118

119

120

121

122

13) What do you understand by parameterized pipeline? OR Your QA team wants to run a Jenkins pipeline but sometimes with different input values (like environment = dev/qa/prod, or version = v1.2). How would you design a pipeline so that it accepts parameters before execution?

```
pipeline {
  agent any
  parameters {
    choice choices: ['DEVELOPMENT', 'STAGING', 'PRODUCTION'],
    description: 'Choose the environment for this deployment.',
    name: 'ENVIRONMENT'
  }
}
```

Answer:



A parameterized pipeline allows you to pass values like branch name, environment, or build options at runtime. You define parameters at the top of Jenkinsfile using "parameters" block. It makes pipelines reusable and interactive.

14) What is Blue Ocean in Jenkins?

Answer:

116

117

118

119

120

121

122

123

Blue Ocean is a modern UI for Jenkins. It provides a clean visualization of pipelines, parallel steps, and stages. It also simplifies pipeline creation using a visual editor and improves overall user experience.

15) How do you fix Jenkins storage full issue?

Answer:

- * Remove old builds using "Build Discarder"
- * Clean "/var/lib/jenkins/workspace" directory.
- * Use "jenkins-cli" to delete old jobs.
- * Add cron jobs for cleanup or integrate with a log rotation mechanism

Delete old builds:
Configure Build Discarder.

Clean workspace:

```
post {
  always {
    cleanWs()
  }
}
```

Use cleanWs() post build.

Archive less artifacts:
Avoid large files in archiveArtifacts.

Move Jenkins home:
To a larger disk volume.

Log rotation:
Limit console log retention.

16) Jenkins slaves not coming up — what could be the issue?

Answer:



117

118

119

120

121

122

123

124

Common issues:

- Agent not reachable (network or SSH issues)
- Wrong credentials
- Agent Java not installed
- Disk full or high CPU/memory on the agent. Check Jenkins logs and agent logs for detailed errors.

17) How do you call variables using "env" block in Jenkins?

Answer:

```
pipeline {
  environment {
    MY_VAR = 'Hello'
  }
  stages {
    stage('Print') {
      steps {
        echo "${env.MY_VAR}"
      }
    }
  }
}
```

▣ Declare inside environment {}.

▣ Access using env.VAR_NAME

▣ Used in steps, script, etc.

18) How to ensure different slave types for different branches?



Search



ENG



12:17

17-05-2026

118

119

120

121

122

123

124

Answer:

```
stage('Build Main') {
  when { branch 'main' }
  agent { label 'ubuntu-agent' }
  steps {
    echo 'Running on Ubuntu'
  }
}

stage('Build Dev') {
  when { branch 'dev' }
  agent { label 'centos-agent' }
  steps {
    echo 'Running on CentOS'
  }
}
```

☒ Use when + agent per stage.
☒ Define label in agent { label '...' }.
☒ Match branch with when { branch '...' }.

19) How do you handle conditional execution in pipeline?

Answer:

```
stage('Deploy') {
  when {
    branch 'main'
  }
  steps {
    echo 'Deploy to Prod'
  }
}
```

☒ Use when block inside stage.

119

120

121

122

123

124

125

Supports branch, expression, env, equals, etc.

Use script { return true/false } for complex logic.

You can also use custom conditions or environment checks.

20) What do you understand by POST block in Jenkins?

Answer:

```
post {
  always {
    echo 'Cleanup'
  }
  success {
    echo 'Build succeeded'
  }
  failure {
    echo 'Build failed'
  }
}
```

The "post" block is used to define actions that should run after a stage or pipeline completes. You can use conditions like "always", "success", "failure", "aborted" to perform cleanups, send notifications, or archive artifacts.

Runs after stage or pipeline.

Conditions: always, success, failure, aborted, unstable.

Linux and Git Interview Questions & Answers

1) What do you understand by "git fork" command?

Answer:

120

121

122

123

124

125

126

127

Technically, "git fork" is not a Git command but a GitHub feature. When you fork a repository, you create a copy of someone else's GitHub repo under your account. It allows you to make changes independently and optionally create pull requests to contribute back. It's commonly used in open-source collaboration.

2) What is the use case difference between "git fetch" and "git pull"?

Answer:

- "git fetch" downloads commits, branches, and tags from the remote repo without modifying your local working directory. It updates your remote-tracking branches.
- "git pull" does "git fetch" + "git merge", meaning it brings remote changes and merges them into your current branch.
- Use Case: Use "fetch" when you want to inspect or review remote changes first. Use "pull" when you're ready to sync and merge remote changes directly.

3) How do you push local code changes to GitHub?

Answer:

(Commands only)

- git add .
- git commit -m "your message"
- git push origin <branch-name>

4) How can you reverse the previous commit?

Answer:

To remove the last commit [whether it is pushed or unpushed] from local git history without losing changes (keep staged files):

"git reset --soft HEAD~1"

To remove the last commit [whether it is pushed or unpushed] from local git history and discard all changes from local (irreversible):

6) How to save local changes without pushing while switching branch?

Answer:

- Use stash to save changes temporarily --- "git stash"
- git checkout <other-branch>
- To apply changes back -- "git stash pop"

7) How do you troubleshoot slow performance on a Linux server?

Answer:

- Check CPU: "top", "htop", or "mpstat"
- Check memory: "free -m", "vmstat"
- Check disk I/O: "df -h", "iostat", "iotop"
- Check logs: "/var/log/syslog", "/var/log/messages"
- Check load average: "uptime"
- Investigate heavy processes: "ps aux --sort=-%cpu"

Focus on identifying if it's CPU, memory, disk I/O, or network bottleneck, then go deeper with relevant tools.

8) How do you handle permission error on Linux server?

Answer:

- Check file permissions: "ls -l"
- Change file owner: "sudo chown user:group file"
- Change permission: "chmod" (e.g., "chmod 755 file")
- Use "sudo" if you need elevated access
- Make sure user has access to directory path ("x" permission)
chmod +x filename

Common fix:

```
"sudo chown -R ubuntu:ubuntu /path/to/dir"
```

```
chmod -R 755 /path/to/dir"
```

9) Execute a script at a specific time daily — how?

Answer:

Each cronjob follows this format:

```
min * * * * path/to/script.sh
```

Example: Run every day at 3:30 PM

Since cron uses 24-hour format, 3:30 PM is 15:30.

So the cronjob entry would be:

```
30 15 * * * /path/to/your/script.sh
```

Use cron jobs:

1. Open crontab:
Command: "crontab -e"
2. Schedule your script (e.g., at 3:30 PM daily):
30 15 * * * /path/to/your/script.sh
For 2 am, 0 2 * * *
Every minute, */15 * * *
Every hour 0 * * *
3. Make sure the script is executable:
"chmod +x script.sh"
4. Save the crontab and it will run automatically daily.

10) What are the commands to check CPU, memory, disk, and process?

Answer:

- CPU usage: "top", "htop", "mpstat"
- Memory usage: "free -m", "vmstat", "top"
- Disk usage: "df -h", "du -sh *", "lsblk"
- Processes: "ps aux", "top", "htop", "pgrep", "kill"

123

124

125

126

127

128

129

130

```
ec2-user@ip-172-31-9-150: ~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        4.0G  0  4.0G   0% /dev
tmpfs           2.0G  0  2.0G   0% /dev/shm
tmpfs           791M  472K  791M   1% /run
/dev/xvda1      16G   15G  1.5G  91% /
tmpfs           2.0G  0  2.0G   0% /tmp
/dev/xvda128    10M  1.3M  8.7M  13% /boot/efi
tmpfs          391M  0  391M   0% /run/user/1000
```

11) What is git rebase command?

Answer:

is used to **move or combine a sequence of commits** to a new base commit. It's mostly used to:

- Keep history linear and clean
- Update your feature branch with the latest changes from main/master

Let's say:

```
mainline:  A---B---C
feature:   A---B---C
```

Here, your feature branch started from commit A. You made commits B and C. Now someone else updated main:

```
main:      A---B---D---E
```

To bring your feature branch up-to-date with the new main's commits (D and E) instead of merging:

☒ If you do **git merge main**:

```
main:      A---B---D---E
feature:    A---B---C
              |
              +--- merge commit --->
```

☒ If you do **git rebase main**:

```
mainline:  A---B---D---E
feature:    A---B---C
              |
              +--- D --->
              +--- E --->
```

Your branch will look like as if you had created it after E, keeping it linear:

```
git checkout feature
git fetch origin
git rebase origin/main
```

This replays your commits (B, C) on top of the latest main.

12. What does \$? Mean in linux?

Answer:

125

126

127

128

129

130

131

132

In a **Linux/Unix shell**, the special variable **\$?** gives you the **exit status of the last command** that was run.

What does it mean?

- `$? = 0` -- the last command **succeeded** (ran without error)
- `$? != 0` -- The last command **failed** (some kind of error occurred)

Example:

```
bash
$ ls
file1.txt file2.txt
$ echo $?
0
# success (exit code 0)
```

13) What is Artifact Versioning in CI/CD?

- It means **tagging every build output** with a unique version.
- Prevents **overwriting old builds** or confusing deployments.
- Version formats: `v1.0.0`, `build-$(date +%Y%m%d-%H%M)`, etc.
- Helps with **rollback and traceability**.

Example:
In Jenkins:

```
env.BUILD_TAG = "v1.0.${BUILD_NUMBER}"
```

14. Print top 10 cpu consuming processes

```
ps -eo pid,ppid,cmd,%cpu,%mem --sort=-%cpu | head -11
```

```
ec2-user@ip-172-31-9-150:~$ ps -eo pid,ppid,cmd,%cpu,%mem --sort=-%cpu | head -11
PID    PPID  CMD                                %CPU  %MEM
2886   1    /usr/bin/containerd               0.6   1.1
1      0    /usr/sbin/crond/systemd --       0.0   0.4
2      0    [kthreadd]                        0.0   0.0
3      2    [rcu_gp]                          0.0   0.0
4      2    [rcu_per_gp]                      0.0   0.0
5      0    [kublet_kubernetes]              0.0   0.0
6      2    [netns]                           0.0   0.0
8      2    [worker/8:00-eventd_holpr]        0.0   0.0
10     2    [m_jreps_w2]                      0.0   0.0
11     2    [rcu_tasks_kthreadd]              0.0   0.0
```

15 You have a private server that can only be accessed through a bastion host. How do you configure your SSH so that you can directly connect to the private server with a simple `ssh Yabz_demo_vm` command? OR How will you use proxy jump? By staying at your machine, you need to connect to a target server [private ec2] which is reachable

by a bastion in between by using proxy jump?

```
Host bastion
HostName 5.188.107.71
Port 22
User manish
IdentityFile ~/.ssh/id_rsa.pem    #your pvt key

Host private server
ProxyJump bastion
User manish
HostName 172.31.135.83
IdentityFile ~/.ssh/id_rsa.pem
IdentitiesOnly yes
```

1 create config file under .ssh

2 inside config file, configure bastion and target server details like above by giving your private key.

16) Logrotation for nginx?

Add this file in /etc/logrotate.d

```
/var/log/nginx/*.log {
    daily
    missingok
    rotate 14
    compress
    delaycompress
    notifempty
    create 0640 www-data adm
    sharedscripts
    prerotate
        if [ -d /etc/logrotate.d/httpd-prerotate ]; then \
            run-parts /etc/logrotate.d/httpd-prerotate; \
        fi \
    endscript
    postrotate
        invoke-rc.d nginx rotate >/dev/null 2>&1
    endscript
}
```

Then in /var/log/nginx, logs will be rotated like,

```
-rw-r----- 1 www-data adm 19K Sep 1 12:39 access.log.10.gz
-rw-r----- 1 www-data adm 21K Sep 4 06:24 access.log.9.gz
-rw-r----- 1 www-data adm 20K Sep 5 06:25 access.log.8.gz
-rw-r----- 1 www-data adm 29K Sep 5 18:29 access.log.7.gz
-rw-r----- 1 www-data adm 11K Sep 7 06:24 access.log.6.gz
-rw-r----- 1 www-data adm 8.7K Sep 8 06:24 access.log.5.gz
-rw-r----- 1 www-data adm 14K Sep 8 10:25 access.log.4.gz
-rw-r----- 1 www-data adm 18K Sep 9 09:31 access.log.3.gz
-rw-r----- 1 www-data adm 17K Sep 11 06:24 access.log.2.gz
-rw-r----- 1 www-data adm 654K Sep 12 06:24 access.log.1
-rw-r----- 1 www-data adm 670K Sep 12 13:15 access.log
```

Helm Interview Questions & Answers

1 What is a Helm Chart, and what are its main components?

Answer:

A **Helm Chart** is a collection of files that describe a set of Kubernetes resources. The main components of a Helm Chart include:

- **Chart.yaml:** Contains metadata about the chart, like the name, version, and description.
- **values.yaml:** A default configuration file with customizable parameters that users can override during installation.
- **templates/:** A directory containing YAML templates for Kubernetes resources (e.g., Deployments, Services, ConfigMaps). These templates use Go templating language to allow dynamic configurations.
- **charts/:** An optional directory containing dependencies, allowing charts to reference other charts.

These components help package applications and define how they should be deployed in Kubernetes.

2 How do you install a Helm Chart?

Answer:

```
helm install my-release my-chart
```

In this command, **my-release** is the name of the release, and **my-chart** is the name of the chart you want to install. Helm retrieves the chart from a repository, installs it into the Kubernetes cluster, and gives it a release name. You can also customize configuration values during installation by using `--set` or a custom values file with `-f`.

3 Why is helm used?

Answer: Helm is a **package manager for Kubernetes**. In bare k8, we need to apply many deployment, service, ingress yaml files. Helm treats an application as a whole and installs it as chart with values file. Helm creates templates for different objects and we can supply values to them from outside from values file and all different components can be controlled as under one chart.

Helm uses GO templates. It works such that values are supplied from values file to placeholders in different templates of deployment service, etc inside templates.

128

129

130

131

132

133

134

135

Helm uses:

- Go templates

example:

```
or
replicas: {{ .Values.replicaCount }}

values.yaml:
replicaCount: 3
```

At install time, Helm replaces the value.

4 What is a Helm Release?

Answer:

A release is:

- A running instance of a chart in a namespace

Example:

```
helm install myapp ./mychart
```

Here:

- Chart = mychart
- Release = myapp

You can install the **same chart multiple times** with different release names.

5 What are Helm hooks?

Answer:

Helm hooks are scripts that allow you to trigger actions at specific points in a chart's lifecycle, like before or after installing, upgrading, or deleting a chart.

Hooks allow you to:

- Run actions at specific lifecycle events

Examples:

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

File View Tools Help

Share

129

130

131

132

133

134

135

136

- pre-install
- post-install
- pre-upgrade

Use cases:

- DB migration
- Job before deployment

6 How do you manage different environments using Helm?

Common ways:

- Separate values-dev.yaml, values-prod.yaml
- Use:

```
helm upgrade myapp ./chart -f values-prod.yaml
```

7 What are some best practices for creating Helm Charts?

Answer:

- Minimise hard coded values
- Give proper versions to charts
- Use helm hooks wisely.
- Use clear naming.

8. How do you manage helm releases?

Answer: Releases can be managed with commands like:

- helm list: Lists all active releases.
- helm upgrade: Updates an existing release to a new chart version or configuration.
- helm rollback: Reverts a release to a previous version if issues occur.
- helm uninstall: Removes a release from the cluster.

9 How will you debug and check you helm templates?

Answer: For validation, we can use dry run command. It does not deploy anything actually, only for validating templates.

```
helm install myapp ./chart --dry-run --debug
```

130

131

132

133

134

135

136

16. Any way can u check syntax issues in helm?

Answer:
We can use `helm lint`

It checks:

- Chart structure
- Syntax issues
- Best practices

Example:
`helm lint ./mychart`

11How do you manage different environments using Helm?

Answer: Use Separate values-dev.yaml, values-prod.yaml like
`helm upgrade myapp ./chart -f values-prod.yaml`

12 How will you handle dependency in helm charts?

Answer: Helm dependency is a chart that another chart depends on.
Can be managed by adding it in chart.yaml

```
dependencies:  
- name: mysql  
  version: 9.10.0  
  repository: https://charts.bitnami.com/bitnami
```

13 How will you debug in real production Helm issue ?

Problem: Upgrade failed, app not starting
Fix:

- helm rollback
- Check helm history
- Debug with `--dry-run --debug`

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

File View Tools Help

Share

Generic Interview Questions & Answers

1 What are the tools and Technologies that you use in your devops project?

Answer:

Cloud: AWS

Version Control System: We are using GitHub/GitLab repository to store our application code.

For CICD : we are using Github actions workflows/ Jenkins pipelines / Gitlab CICD .

For Containerization: we are using docker, we have dockerfile written for our applications placed at github repo beside our code at root folder.

To automate the infrastructure Provisioning: we are using Terraform. We have standard terraform modules written for major resources like ec2, vpc, rds, etc.

We are using Helm/Flux for managing the k8 layer for our applications.

We are tracking our tickets by JIRA/email/slack.

We get alerts on Pagerduty/email/slack.

We have Dynatrace Grafana ELK for monitoring tool.

2 Any cost optimization activity that you have implemented in your system?

Answer: We have added auto start-shut down for staging dev qa environments, so the servers get shut down in the night and reboot in the morning. Saving cost to us.

3 Tell me about your current projects.

Answer: I have worked on a project which is ITSM tool and it's very much equivalent or you can say competitor to ServiceNow tool. There are 26 microservice including SLA, Foundation, Main, Scheduler, Notification, Rules etc. All these services are deployed over AWS EKS and to do the deployment I'm using Jenkins pipeline.

132

133

134

135

136

137

138

139

To create complete infrastructure over AWS Cloud, We have used TERRAFORM modules and this terraform is also being deployed using JENKINS pipeline only. All these microservices are being deployed on Kubernetes system so for that we're using HELM as a package manager and Docker to create image out of source code.

At the end, we monitor our system using Grafana, pagerduty to keep our services 24*7 available.

I also worked on **Kubernetes deployments**, configmaps, secrets, and Helm charts. The main focus was automating infra, managing CI/CD, and ensuring services run smoothly.

4) Tell me about yourself.

Answer: I have 4 years of experience as a devops engineer. I have started my career in Technical support. I worked in production support role for about 1.5 years. During that time, I developed a strong understanding of troubleshooting real-time issues in live environments. And then I upgraded my skillset and switched to devops.

I've been working primarily on **AWS cloud**, with hands-on experience in commonly used services like ec2, vpc, rds, route53, s3, iam, lambda, etc.

I have used Jenkins / GitHub actions for cicd, terraform as IAC TOOL, Docker and k8 also.

Then if needed add your roles and responsibilities.

In my current company I am working on writing terraform modules to automate our infra creation.

OR Can add these points if needed.

In my current role, I'm responsible for:

- Writing reusable **Terraform modules** to automate infrastructure provisioning.
- Managing **CI/CD pipelines** for multiple environments (dev, staging, prod).

133

134

135

136

137

138

139

140

- Handling **production deployments, hotfixes, and environment upgrades.**
- Working closely with the development and support teams to **resolve issues, reduce downtime,** and improve system reliability.
- Adding **monitoring and alerting** using tools like CloudWatch, Prometheus, and setting up Stack/email notifications.
- Occasionally contributing to **POCs** and internal automation tools to improve efficiency.

I'm always looking to improve system stability, reduce manual effort through automation, and ensure faster and safer delivery of code to production.

Note: Frame your skills according to JD.

5 | What are your day to day activities look like? How you start your day?

Answer:

Note: Frame your answer according to your tasks. These are just sample activities that you can mention.

I usually start my day by checking the Jira board to see if any new tickets are assigned to me. My tasks vary but mainly include:

1. **Handling deployments, upgrades, or hotfixes** for specific customers or environments.
2. **Investigating issues** reported by the support team, like 504 or 404 errors, or service downtime. First, we focus on restoring the service quickly. Once it's stable, we prepare an RCA to identify the root cause and how to prevent it in the future.
3. **Working on access-related requests** — like creating new users or giving read-only DB access.
4. **Troubleshooting CI/CD pipeline failures** — checking whether the issue is in the pipeline or due to some application-side error (e.g., a Docker image failed to build due to a missing dependency). If it's a code issue, we raise a ticket for the development team.
5. Creating or modifying CI/CD pipelines for new environments like QA, staging, or feature branches.

134

135

136

137

138

139

140

141

6. Adding monitoring alerts — such as setting up email or Slack alerts for high CPU, RAM, or disk usage.

7. Occasionally working on POCs or automation tasks to improve processes.

Sometimes we also support teams during releases or environment migration activities.

6) Can you walk me through your end-to-end CI/CD pipeline setup?

or "How do you handle different environments like staging and production in your CI/CD setup?"

Answer: Expected answer: QA team tests; if approved, dev raises PR to merge develop → main; manual approval leads to prod deployment

Answer: Explain Git → Jenkins → Docker → ECR → Helm → EKS → staging/prod namespaces

your end-to-end CI/CD process using Jenkins, ECR, Helm, and EKS (with staging and prod namespaces in the same cluster);

CI/CD Flow (Jenkins + GitHub + AWS EKS)

```
graph TD
    1[1. Developer pushes code to 'develop' branch (GitHub)] --> 2[2. Jenkins CI triggered via webhook]
    2 --> 3[3. Jenkins Pipeline:  
- Build the code  
- Run unit/integration tests  
- Build Docker image  
- Tag image (e.g., v1.0.1 or 20250708-1234)  
- Push image to AWS ECR]
    3 --> 4[4. Jenkins CD stage:  
- Use Helm to deploy image to EKS to 'staging' namespace]
    4 --> 5[5. QA Team tests app in staging]
    5 --> 6[6. If approved -> Dev team creates PR: 'develop' -> 'main']
    6 --> 7[7. Manual approval + merge to 'main']
    7 --> 8[8. Jenkins Prod Pipeline triggers (from 'main' branch)  
- Rebuild image (optional) or pull from ECR  
- Helm deploy to EKS to 'production' namespace]
```

Here's the **standard directory structure** for a project using **Jenkins, Docker, Helm, and CI/CD for staging & prod**, where you:

- Build the app
- Use Dockerfile for containerization
- Use Jenkinsfile for pipeline
- Use Helm for deployment

136

137

138

139

140

141

142

143

Recommended Directory Structure

```
perl
my-app/
├── src/                # App source code
│   ├── main.py / index.js  # Entry point
│   └── ...
├── Dockerfile          # For building app image
├── Jenkinsfile         # CI/CD pipeline steps
├── charts/             # Helm charts directory
│   └── my-app/         # Actual chart name
│       ├── Chart.yaml
│       ├── values.yaml  # default values (used in staging)
│       ├── values-prod.yaml # overrides for production
│       └── templates/
│           ├── deployment.yaml
│           ├── service.yaml
│           └── ingress.yaml (optional)
│           └── ...
├── helmfile.yaml (optional) # If using helmfile for multi-env
└── README.md
```

Where is each component?

File/Folder	Purpose
Dockerfile	Defines how the Docker image is built
Jenkinsfile	Contains CI/CD pipeline logic
charts/my-app/	Helm chart directory for deployment
values.yaml	Default values (e.g., used for staging)
values-prod.yaml	Overrides for production deployment
templates/	Helm templates like Deployment, Service, etc.

137

138

139

140

141

142

143

144

File/Folder	Purpose
src/	Actual application code

7) How do you keep yourself updated with latest tools and technologies?

Answer: LinkedIn, O'Reilly, Communities, Documents etc

8) Why should we select you ?

Answer: Skills, certifications, your skills acc to JD, team handling, leadership capabilities etc.

9) What is the most challenging thing you faced in your project?

Answer: The most challenging thing was to do migration from Cloudformation to Terraform for Infrastructure management.

Initially when we started working with AWS Cloud, the complete infrastructure was setup using CloudFormation templates with the help of PS team of AWS. After one year, We decided to have centralized way of managing infrastructure and we thought of adopting terraform. We started with importing statefiles for already existing infrastructure. We developed terraform import script for each customer according to customer environment one by one and ran these scripts using Jenkins pipeline.

Once migration was done and state file was stored in S3, now we keep enhancing and managing our infrastructure using Terraform only. This was a challenging task as we had to do smooth migration of real production infrastructure.

10) If deployment fails - how are you going to troubleshoot and roll back?

Answer: If deployment fails, your rolling update strategy will ensure that no new pod will come up with newer version. But in case if you have to rollback, you can simply deploy your older version images of application which is compatible with all microservice.

To troubleshoot failed deployment you can check pod logs, events etc.

Command to use:

```
kubectl logs podname
kubectl get events
Kubectl describe pod/podname -n namespace
```

138

139

140

141

142

143

144

145

11) How to Secure our overall application?

Network layer - use private subnets for ec2 and rds.

Makes sure in security group of your rds and Elastic cache redis or any database keep only the vpc cidr of the vpc in which u have your applications only.

2 IAM Create separate iam role for each. Create centralised groups and roles for different people. Give least permission roles.

Data security - use kms secret manager in aws to store db credentials, etc. Rotate keys regularly.

Application layer

Monitoring, logging and alerts

Backup and recovery - enable rds backup snapshots and ebs snapshots so that we restore in case of accidental deletion.

Implement activity tracker on critical servers that actually logs history of commands run by any user on that server.

Scenario/Troubleshooting Interview Questions & Answers

1) Developer creates jira or ping directly that this server is not connecting or it's not reachable.

Answer: We'll get into AWS console and will check whether health check is passed or not for that server. If CPU and memory was consumed highly due to which this server has been stuck so we'll reboot/restart the server. Once we as DevOps Team able to connect to that server then we'll check CPU, RAM and Disk.

Along with all this, we will perform basic checks as well, we'll see whether that their user is able to login on the server or not. Any port they are using to access the server got changed like port 22 got disabled/removed by the audit team for security purpose. We will check/add it. If needed we will change the ssh port for this server like 922, etc.

2) Customer application is generating too much log someday when there is a campaign or festive season and storage on the server was getting consumed frequently. Developers are reporting to clean up and make enough resource to store logs. What will you do ?

139

140

141

142

143

144

145

146

Answer: First we will free some space by deleting unused logs in /var/log then if not there we will implement log rotation so that whenever the current log file size reaches up to 1 GB the data of logs file will be moved to another file and it will be zipped so that actual log file will be free once again to store new logs. And previous zipped files occupying less space on server.

3) We had created one Jenkins pipeline which was being utilised to create image from application source code & Dockerfile , Push the image to ECR and deploy it over Kubernetes cluster. While running this pipeline, it was throwing an error that user/pipeline doesn't have access to run Dockerfile.

Answer: We started troubleshooting and found that my Jenkins pipeline was running through Jenkins user which is very obvious, but this Jenkins user was not added in Docker Group. We added Jenkins user in Docker Group and after that it started running pipeline smoothly. The learning here was that Jenkins user was to be added in Docker group then only it will allow execution of pipeline.

The command what we used to add user in Docker Group:

```
sudo usermod -aG docker $USER
```

4) I was running Jenkins pipeline to deploy application upgrade. Application was being upgraded from 10.0 version to 12.0 version. Pipeline thrown an error that no storage is left on device.

Answer: To troubleshoot this, we logged into the AWS EC2 where Master Jenkins was running and I went into this /var/lib/workspace directory over server. There I checked for all temporary files and extra workspace which are created and then I deleted all these files and folder and we got some free space to run our pipeline.

To prevent from such issues, we also added post block in our Jenkins pipeline so that from next time, all temporary files will be cleaned up once pipeline is executed till the end.

```
post
{
    //Clean after build
    always
    {
        clearWs()
    }
}
```


141

142

143

144

145

146

147

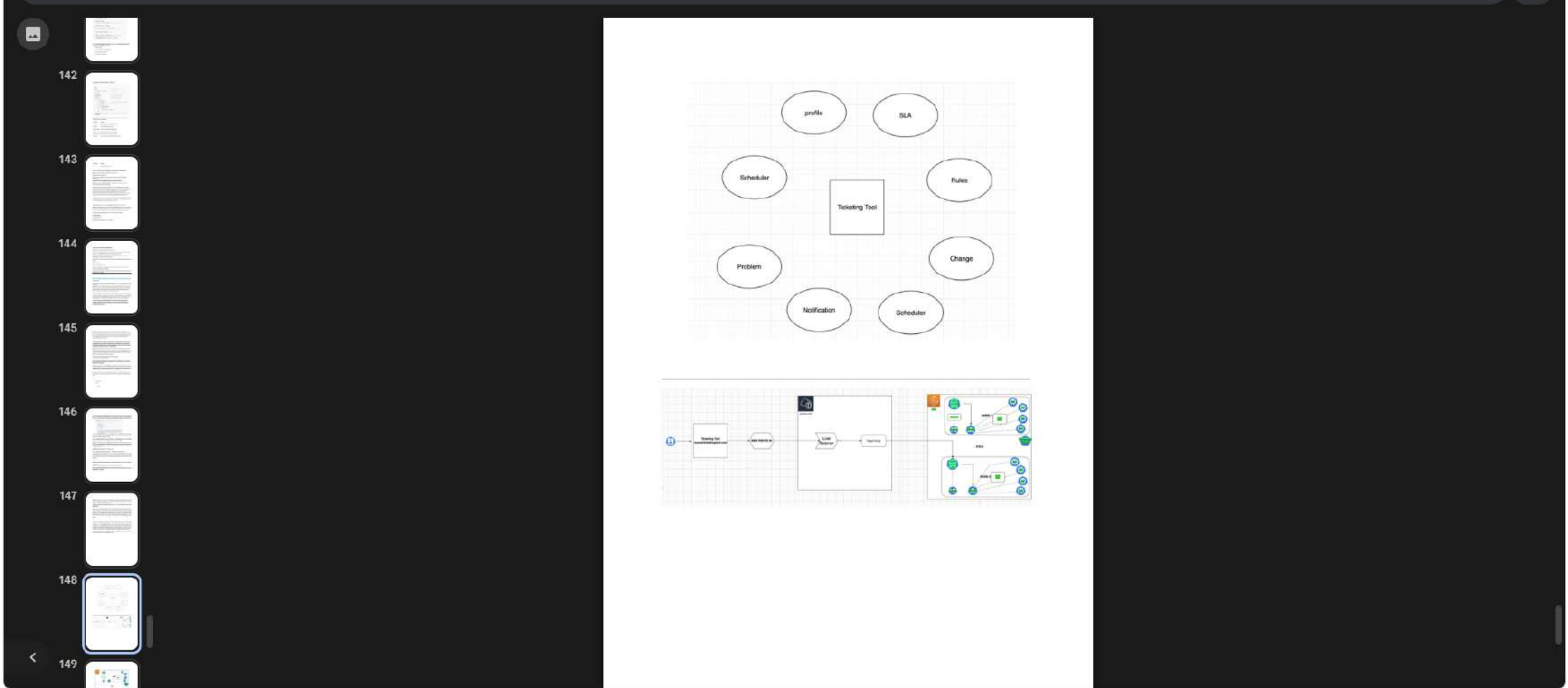
Answer: Yes for each customer, we manage separate AWS accounts. Within this AWS account complete infrastructure is provisioned including separate env. For each customer, we have different AWS account.

10) Can you explain the overall flow of the project—from code commit to production deployment?

Answer: First of all, Developer develop code on local system means visual studio code. Once we're done with code writing, we push it to github repository using git commands (git status, git add .., git commit -m "custom message ", git push). Once code changes are pushed to repo, it triggers Jenkins pipeline using git webhook. Next all steps will be handled by Jenkins pipeline. Now, This Jenkins pipeline will create image out of this source code and Dockerfile. Once image is created, we push it to AWS ECR using Jenkins pipeline.

After this, if we have to deploy these changes for production, we create change ticket which goes for different approval from our team and customer side as well. Once approved, we run our CD Jenkins pipeline for next step which is deployment. This step will connect to customer cluster and it will do all deployment. Once Deployment is successful, we monitor their availability whether all microservices are up or not. Once everything is up and running, we pass it to QA team for validation/Functional testing.

Apart from this, We also have monitoring setup using Grafana, where we monitor all our production applications and staging applications.



143

144

145

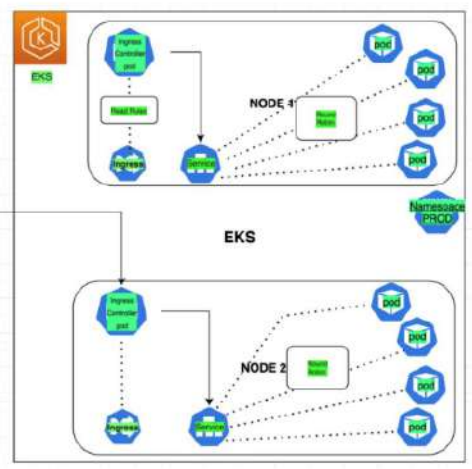
146

147

148

149

150



11) How will you give access to developer on an ec2 machine without sharing ec2 pem key?

Answer:

First from developer, ask to generate his/her key pair on his machine. He will keep the pvt key with himself and share his public key with us to create his user on EC2. Command to create key-pair on linux: `ssh-keygen -t rsa`

Command on windows [to create pem key format] is:

`ssh-keygen -t rsa -b 2048 -m PEM -f 'file-name-you-want-with-path'`

example,

11-08-2025 23:08 1,679 rinki-key

11-08-2025 23:08 415 rinki-key.pub

Now you take this .pub key contents from the developer and can use it for user creation on the ec2 machine. Add it to authorized_keys inside .ssh folder for the new user.

Steps to create user on desired ec2-machine

```
ec2-user@ip-172-31-9-150 home$  
ec2-user@ip-172-31-9-150 home$ sudo adduser rinki  
ec2-user@ip-172-31-9-150 home$  
ec2-user@ip-172-31-9-150 home$  
ec2-user@ip-172-31-9-150 home$ sudo su - rinki  
rinki@ip-172-31-9-150 ~$  
rinki@ip-172-31-9-150 ~$ ls -la  
total 12  
drwxr-xr-x  2 rinki rinki 64 Aug 11 17:18 .  
drwxr-xr-x  5 root root 4096 Aug 11 17:18 ..  
-rw-r--r--  1 rinki rinki 18 Jan 28 2023 .bash_logout  
-rw-r--r--  1 rinki rinki 141 Jan 28 2023 .bash_profile  
-rw-r--r--  1 rinki rinki 492 Jan 28 2023 .bashrc  
rinki@ip-172-31-9-150 ~$  
rinki@ip-172-31-9-150 ~$ mkdir .ssh  
rinki@ip-172-31-9-150 ~$ cd .ssh/  
rinki@ip-172-31-9-150 .ssh$ vi authorized_keys  
rinki@ip-172-31-9-150 .ssh$  
rinki@ip-172-31-9-150 .ssh$ chmod 600 authorized_keys  
rinki@ip-172-31-9-150 .ssh$  
rinki@ip-172-31-9-150 ~$ cd ..  
rinki@ip-172-31-9-150 ~$ chmod 700 .ssh/  
rinki@ip-172-31-9-150 ~$
```

Now developer can ssh to ec2-machine from his laptop by command

ssh -i pem-key-poth user@ip

ssh -i C:\Users\rinki.rathore\Documents\rinki-key rinki@13.233.121.218

Some more troubleshooting Interview Questions

1. A service isn't starting on a Linux server. How do you debug it?

* Check status with 'systemctl status'

145

146

147

148

149

150

151

152

* Check logs using `journalctl -xe`

* Verify config syntax

* Check if required port is free

2. CPU usage is at 100% on a production server, what steps will you take?

* Run `top` / `htop` to identify process

* Check for infinite loops or runaway processes

* Review logs of heavy processes

* Restart or optimize service if needed

3. Disk space suddenly fills up on your EC2 instance — how do you fix it?

* Run `df -h` for usage

* Use `du -sh *` to locate large dirs

* Clear old logs, temp files

* Set up log rotation

4. A container keeps restarting (CrashLoopBackOff). How do you troubleshoot it?

* `kubectl describe pod` for events

* Check `kubectl logs`

* Verify config, env vars, image

* Check resource limits

5. Your container can't connect to the internet. What could be the issues?

* Check network mode/bridge

146

147

148

149

150

151

152

153

- * Verify DNS resolution inside container
- * Check host firewall rules
- * Restart Docker network

6. Logs aren't showing up from a running container. How do you debug?

- * Use `docker logs <container>`
- * Check logging driver configuration
- * Ensure app writes to stdout/stderr
- * Inspect log rotation settings

7. A Pod is stuck in Pending state. How do you fix it?

- * `kubectl describe pod` for events
- * Check node resources availability
- * Check node taints/tolerations
- * Verify PVC binding if using volumes

8. A Pod is running but not reachable via Service. What steps do you take?

- * Check if Service selector matches Pod labels
- * Verify Service type (ClusterIP/NodePort/LoadBalancer)
- * Run `kubectl get endpoints`
- * Check NetworkPolicy or firewall rules

9. One node in the cluster is NotReady. How do you debug?

- * `kubectl describe node`

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

File View Tools Help

Page 153 / 158

75%

147

148

149

150

151

152

153

154

- * Check kubelet logs on that node
- * Verify Docker/container runtime status
- * Check network connectivity and disk space

10. You applied a Deployment, but Pods aren't being created. What do you check?

- * `kubectl describe deployment` for events
- * Check YAML syntax/valid API version
- * Verify image accessibility
- * Look for quota or RBAC issues

11. Your Jenkins pipeline suddenly fails after a code merge — how do you approach debugging?

- * Check console output logs
- * Identify failing stage or command
- * Compare working commit vs failing commit
- * Verify build tools/dependencies

12. A job works on one agent but fails on another. What could be wrong?

- * Check agent environment differences
- * Verify installed tools and versions
- * Compare permissions and credentials
- * Ensure consistent workspace setup

13. A deployment stage in your pipeline hangs forever — how would you investigate?

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

File View Tools Help

Page 154 / 158

75%

148

149

150

151

152

153

154

155

- * Check script logs for stuck command
- * Verify network connectivity (to repo, servers)
- * Check for interactive commands blocking CI
- * Add timeout to stages

14. Your EC2 instance is up, but you can't SSH into it. What do you check?

- * Verify security group rules (port 22)
- * Check key pair being used
- * Confirm correct user (ec2-user, ubuntu)
- * Check instance system logs

15. An application in AWS can't connect to RDS. How do you troubleshoot?

- * Verify RDS security group rules
- * Check DB endpoint and port
- * Confirm credentials
- * Ensure app's VPC/subnet has access

16. A website hosted on S3 + CloudFront is showing 403 Forbidden. How do you debug?

- * Check S3 bucket policy and object ACL
- * Verify CloudFront origin settings
- * Check OAI/DAC configuration
- * Ensure correct index document

17. Your app works inside the cluster but not externally. What do you check?

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

File View Tools Help

Page 155 / 158

75%

149

150

151

152

153

154

155

- * Verify Service type (NodePort/LoadBalancer)
- * Check Ingress rules if used
- * Confirm firewall/load balancer rules
- * Check external DNS resolution

18. DNS resolution is failing inside a container. How do you fix it?

- * Check `/etc/resolv.conf` inside container
- * Verify CoreDNS is running in cluster
- * Restart kube-dns/CoreDNS Pods
- * Check node-level DNS settings

19. Load balancer health checks are failing — what's your process?

- * Verify health check path/port matches app
- * Check app logs for readiness issues
- * Confirm SG/firewall allows health check IPs
- * Test endpoint directly with curl

DevSecOps Interview Questions & Answers

1. What is DevSecOps? How is it different from DevOps?

Answer: DevSecOps integrates security practices into DevOps pipelines. It ensures security is part of every stage — from code to production — instead of



150

151

152

153

154

155

156

157

being an afterthought. Examples include static code analysis during build, container image scanning before deploy, and secrets management in runtime.

2. How do you scan your code for vulnerabilities?

Answer: We use static code analysis tools like SonarQube to detect code smells, bugs, and security hotspots. We also use tools like Snyk, Checkmarx, or Fortify to scan known vulnerabilities in dependencies. These tools are integrated into our CI/CD pipelines, and builds fail if vulnerabilities are found.

3. How do you manage secrets in your DevOps pipeline?

Answer: Secrets are never hardcoded in code or pipelines. We use tools like AWS Secrets Manager, SSM Parameter Store, HashiCorp Vault, or Kubernetes Secrets (with encryption at rest) to store them securely. In CI tools like Jenkins or GitHub Actions, secrets are stored using environment variables or credentials stores.

4. How do you scan container images for vulnerabilities?

Answer: We use container scanning tools like Trivy, Aqua Security, Snyk, or Clair. These tools scan Docker images for known CVEs and are integrated into CI/CD pipelines to block vulnerable builds before they are deployed.

5. What is the difference between SAST and DAST?

Answer: SAST stands for Static Application Security Testing, which scans code without executing it. It helps catch code issues early in the **development cycle** and uses tools like SonarQube or Checkmarx. DAST stands for Dynamic Application Security Testing, which scans the running application to find runtime vulnerabilities. Tools used include OWASP ZAP and Burp Suite.

6. How do you implement security in Kubernetes?

Answer: We use RBAC to restrict access and network policies to limit pod-to-pod communication. Containers are not run as root. We implement PodSecurityAdmission or PodSecurityPolicies, encrypt secrets at rest, and scan container image before deployment.

7. How do you enforce security policies in the pipeline?

Answer: We set up quality gates using tools like SonarQube and include security scanning stages in Jenkins or GitHub Actions. The build is configured to fail if high CVEs are found in Docker images or if critical issues are detected in code scans. In Kubernetes, we use tools like OPA Gatekeeper or Kyverno to enforce security policies.



Search



ENG

IN

12:20

17-05-2026

8. What are OWASP Top 10 vulnerabilities?

Answer: The OWASP Top 10 includes the most common web vulnerabilities such as injection (like SQL injection), broken authentication, sensitive data exposure, security misconfiguration, broken access control, and cross-site scripting (XSS). We scan for these vulnerabilities using tools like OWASP ZAP, Burp Suite, and other SAST/DAST tools.

9. How do you handle third-party dependency vulnerabilities?

Answer: We use tools like Snyk, OWASP Dependency-Check, or built-in scanners such as npm audit or pip audit. CI/CD is configured to notify or auto-patch when issues are found. We also lock versions using package managers such as Gemfile.lock or package-lock.json.

10. What is a Software Bill of Materials (SBOM)?

Answer: An SBOM lists all components in a software package, including third-party libraries and dependencies. It helps identify licensing issues, known vulnerabilities, and supply chain risks. Tools like Syft, Anchore, or Snyk are used to generate SBOMs.

11) What is the difference between static and dynamic code analysis? What tools have you used like SonarQube, RuboCop, etc.?

Answer:

Static Analysis	Dynamic Analysis
Done without running the code	Done during execution/runtime
Catches syntax issues, style, bugs	Catches runtime issues, memory leaks
Example: SonarQube, RuboCop	Example: OWASP ZAP, Burp Suite

Static Code Analysis is performed without executing the code. It checks code quality, security vulnerabilities, and coding standards.

Tools like SonarQube, RuboCop, ESLint, or Pylint scan the source code to identify issues early.

Dynamic Code Analysis is done during runtime. It analyzes the application's behavior — like memory usage, security flaws, and performance under real conditions.

PDF icon

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

File View Tools Help

Share

Page 158 / 158

75%

151

152

153

154

155

156

157

158

Tools include OWASP ZAP, Burp Suite, Valgrind, and APMs like New Relic.

12) What is the use of SonarQube works?

Answer: SonarQube checks for code quality using metrics like code smells, bugs, vulnerabilities, code duplication, and test coverage. For example, it reports the percentage of duplicated code and coverage. We use quality gates to ensure our new code has no critical issues, high coverage, and minimal duplication. For Ruby, we integrate RuboCop for linting and SonarQube for overall code quality reporting.

PDF icon

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

File View Tools Help

Share

Page 158 / 158

75%

151

152

153

154

155

156

157

158

Tools include OWASP ZAP, Burp Suite, Valgrind, and APMs like New Relic.

12) What is the use of SonarQube works?

Answer: SonarQube checks for code quality using metrics like code smells, bugs, vulnerabilities, code duplication, and test coverage. For example, it reports the percentage of duplicated code and coverage. We use quality gates to ensure our new code has no critical issues, high coverage, and minimal duplication. For Ruby, we integrate RuboCop for linting and SonarQube for overall code quality reporting.